- Course Code: CSC462 Course
- Title: Artificial Intelligence
- Credit Hours: 4 (3,1)
- Pre-Requisites: CSC102-Discrete Structures
- Textbook: 1. Artificial Intelligence: A Modern Approach, Russell, S., and Norvig, P., Pearson, 2020.
- Reference Book: 1. Artificial Intelligence Basics: A Non-Technical Introduction, Taulli, T., Apress, 2019.

Artificial Intelligence - Search Algorithms

Search Concepts, Problem Formulation, Search Space Definition, Types of Search Algorithms, Uniformed Search, Depth First Search (DFS), and BFS & DFS Comparison.

Artificial Intelligence - Solving Problems by Searching

- **Search:** In **Artificial Intelligence (AI)**, **search** is the process of exploring different possible solutions to a problem in order to find the **best or most optimal solution**.
- It is like **problem-solving** where an AI agent moves step-by-step through a set of possible actions to reach a **goal state**.

Search Concepts

- A search problem can have **three main factors**:
- **Search Space:** Search space represents a set of possible solutions, which a system may have **Start State:** It is a state from where agent begins the search.
- **Goal State :** The desired solution or end point.

Uninformed Search and Informed Search

- Two fundamental approaches in AI for solving problems using search algorithms.

- Uninformed Search
- Informed Search

Uninformed Search (Blind Search)

- Explore search space blindly without problem-specific knowledge.
- Uninformed search **does not use any extra knowledge** about the problem. It only knows:
  - The **start point**
  - The **goal**
  - The possible **steps or actions**
- It explores **every possibility** step-by-step, like **searching in the dark**.

Uninformed Search:

- *Example:*
  Looking for a book in a library **without a catalog**, checking each shelf one by one.

Common Algorithms:

- BFS (Breadth First Search)
- DFS (Depth First Search)

BFS (Breadth-First Search)

- BFS is a graph traversal algorithm that explores nodes **level by level**, starting from a given **source node**. It visits **all neighboring nodes** at the **current depth (or level)** before moving on to nodes at the next depth level.
- BFS is commonly used for traversing or searching tree or graph data structures.

Breadth-First Search (BFS)

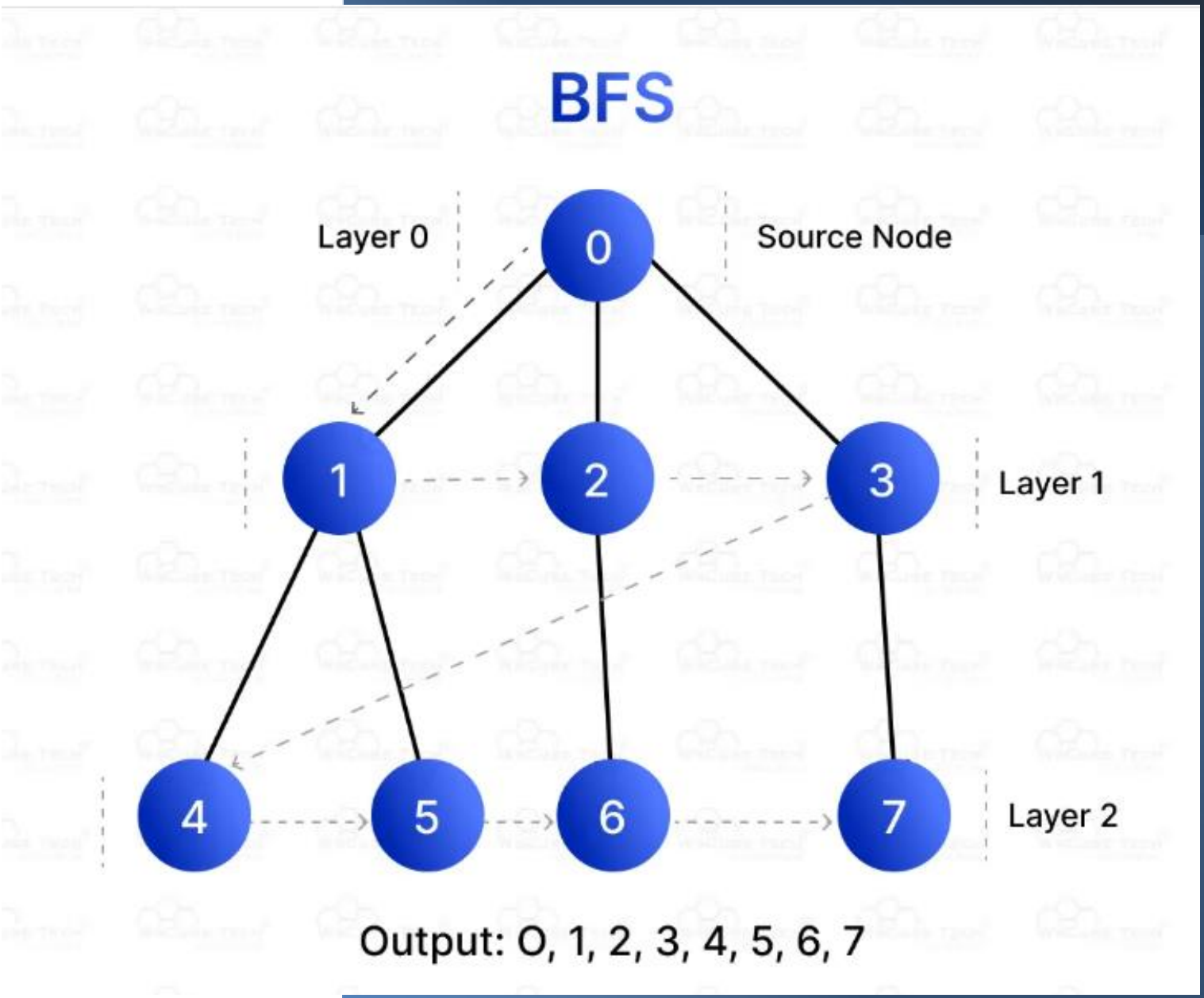
- BFS(Breadth First Search) uses **Queue data structure** for finding the shortest path.
- It works on the concept of **FIFO (First In First Out).**
- **Best for:** Finding the shortest path in an unweighted graph.
- **Shortest Path:** Guaranteed to find the shortest path in an unweighted graph.

Breadth-First Search (BFS)

- **Pathfinding in Weighted Graphs:** Not suitable for weighted graphs.
- **Recursive/Iterative:** Iterative (uses a queue)
- **Graph Representation:** Can be used with both adjacency lists and matrices.
- **Tree Traversal:** Used for level-order traversal in trees.
- **Backtracking:** No backtracking.

Breadth-First Search (BFS)

- **Algorithm Type:** Greedy approach (explores level by level).
- **Graph Type:** Applies to both directed and undirected graphs
- **Time Complexity:** O(V + E)
- **Space Complexity:** O(V) (due to queue storing nodes at the current level)

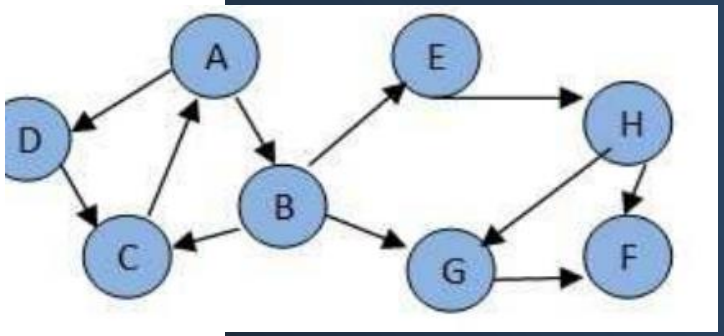

BFS
Layer 0
0
Source Node
1
2
3
Layer 1
4
5
6
7
Layer 2
Output: O, 1, 2, 3, 4, 5, 6, 7

BFS (Example 1)

- **Traversal Order:**
- Starting from node 0:
- Layer 0: Visit node 0 (source node).
- Layer 1: Visit nodes 1, 2, 3 (neighbors of node 0).
- Layer 2: Visit nodes 4, 5, 6, 7 (neighbors of nodes 1, 2, 3).
- Output: 0, 1, 2, 3, 4, 5, 6, 7.

Class Quiz

- Consider the following graph. If there is ever a decision between multiple neighbor nodes in the BFS. Assume we always choose the letter closest to the beginning of the alphabet first.
- **In what order will the nodes be visited using a Breadth First Search?**



How can we get ?

- The answer is: ABDCEGHF
- A->B->D
- B->C->E->G
- D->C
- C->A
- E ->H
- G->F
- H->F->G

What is DFS (Depth-First Search)?

- popular algorithm used for traversing or searching through graph or tree data structure
- It starts at a source node (or vertex) and explores as far along each branch or path as possible before backtracking
- This means it dives deep into the graph before moving to other branches, hence the name "depth-first." DFS can be implemented using either recursion or a stack.

Depth-First Search (DFS)

- **Time Complexity:** O(V + E)
- **Graph Type:** Applies to both directed and undirected graphs
- **Traversal Method:** Explores as deep as possible along one branch before backtracking.
- **Data Structure Used:** Stack (LIFO) or recursion stack.

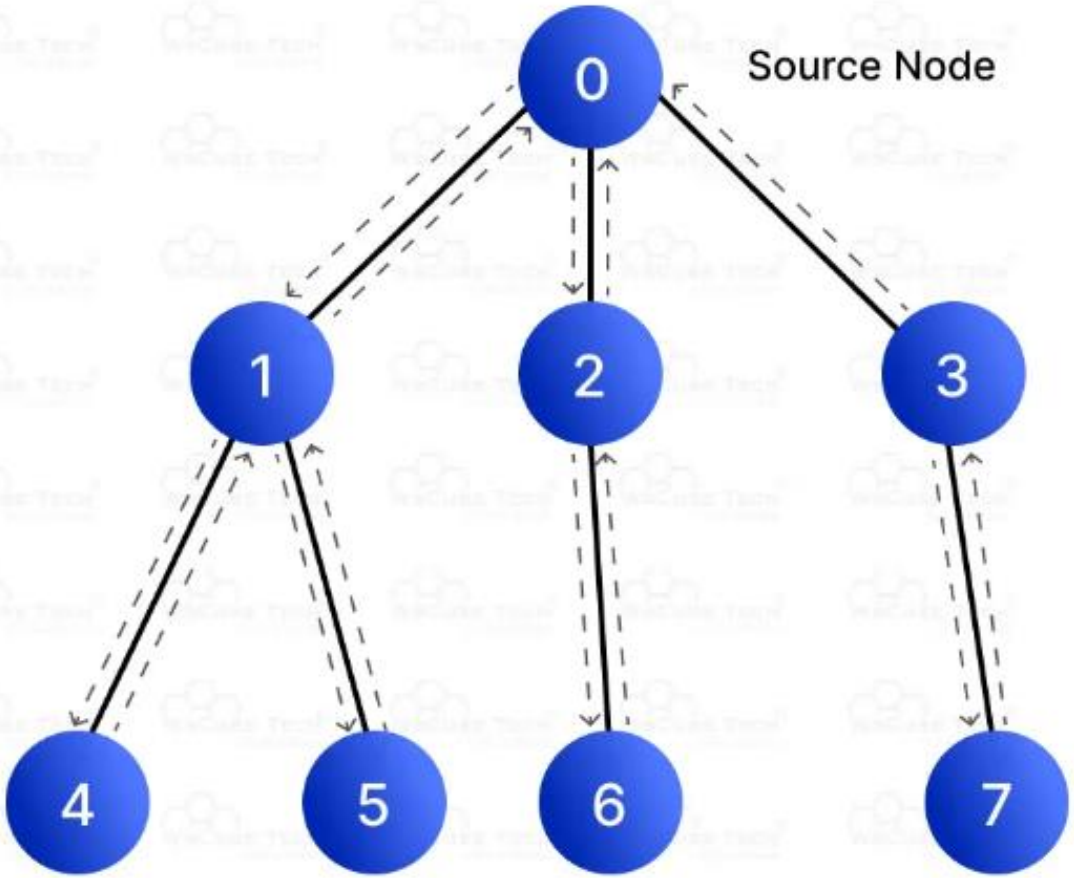
Depth-First Search

- **Space Complexity:** O(V) (due to recursion or explicit stack)
- **Shortest Path:** Not guaranteed to find the shortest path, but explores all paths.
- **Pathfinding in Weighted Graphs:** Not suitable for weighted graphs.
- **Recursive/Iterative:** Can be recursive (via recursion stack) or iterative (using an explicit stack).

Depth-First Search

- **Graph Representation:** Can be used with both adjacency lists and matrices.
- **Tree Traversal:** Used for pre-order, in-order, and post-order traversal in trees.
- **Backtracking:** Uses backtracking to explore new branches.
- **Algorithm Type:** Backtracking approach (explores depth first).

DFS



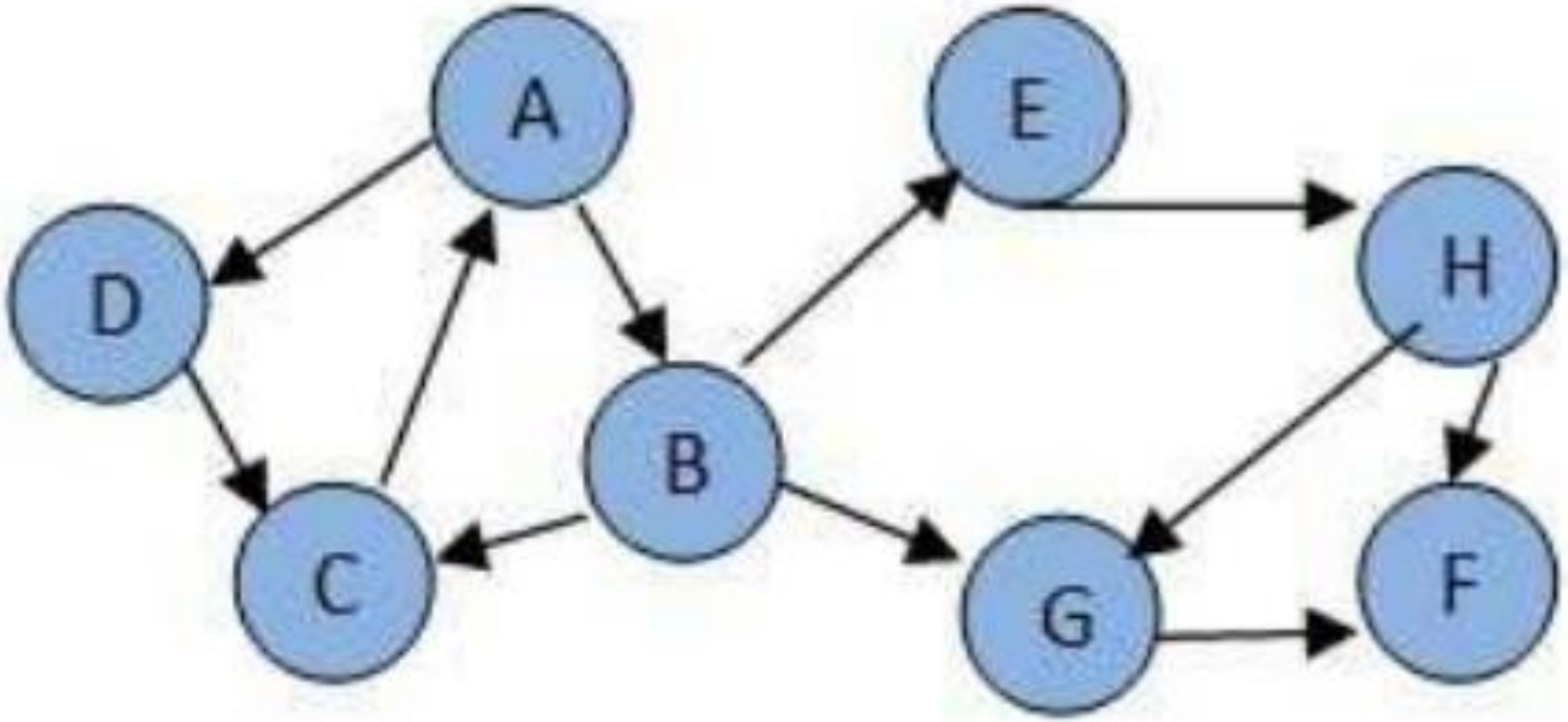
Output: 0, 1, 4, 5, 2, 6, 3, 7

Depth-First Search

- **Traversal Order:**
- Starting from node 0:
- Visit node 0 → Visit node 1 → Visit node 4 (deepest) → Backtrack to node 1 → Visit node 5 → Backtrack to node 0 → Visit node 2 → Visit node 6 → Backtrack to node 2 → Visit node 3 → Visit node 7.
- **Output:** 0, 1, 4, 5, 2, 6, 3, 7

Class quiz

In what order will the nodes be visited using a Depth First Search?



How can we get ?

- The answer is: ABCEHFGD

DFS Applications	BFS Applications
Cycle detection in graphs	Finding the shortest path in unweighted graphs
Finding connected components in a graph	Level-order traversal in trees
Solving puzzles like Sudoku, N-Queens (backtracking)	Web crawling (breadth-first)
Topological sorting in Directed Acyclic Graphs (DAGs)	Social network analysis (finding the shortest path between users)
Finding strongly connected components (Tarjan's/Kosaraju's algorithm)	Finding nodes at a specific level in a graph or tree
Maze solving (exploring all paths)	Minimum moves in a game or puzzle (e.g., 8-puzzle)
Web crawling (depth-first)	Friend suggestion algorithms in social networks
Tree traversal (pre-order, in-order, post-order)	Finding the shortest path in a maze
Pathfinding in games (exploring all paths)	Level-by-level exploration in AI game design

BFS Pseudocode

```
BFS(graph, start):

    # Step 1: Create a queue to keep track of nodes to visit
    queue = []                  # Empty queue
    queue.append(start)         # Enqueue the starting node

    # Step 2: Create a set to keep track of visited nodes
    visited = set()             # Empty set to store visited nodes
    visited.add(start)          # Mark the start node as visited

    # Step 3: Loop until there are no more nodes to explore
    while queue:

        # Step 3.1: Remove the first node from the queue (FIFO)
        node = queue.pop(0)     # Dequeue operation
        print(node)             # Process the current node (e.g., display it)

        # Step 3.2: Explore all neighbors of the current node
        for neighbor in graph[node]:

            # Step 3.3: If the neighbor hasn't been visited yet
            if neighbor not in visited:

                visited.add(neighbor) # Mark neighbor as visited
                queue.append(neighbor) # Enqueue the neighbor for future exploration
```

DFS Pseudocode (Recursive Method)

```
DFS(graph, start, visited):

    # Step 1: Check if the current node has NOT been visited
    if start is not in visited:

        print(start)
        # Process the current node
        # Example: print it, store it, or check if it's the goal node

        add start to visited
        # Mark the current node as visited so we don't visit it again
        # This prevents infinite loops in case of cycles in the graph

        # Step 2: Explore each neighbor (connected node)
        for each neighbor in graph[start]:
            DFS(graph, neighbor, visited)
            # Recursively call DFS for each neighbor
            # This goes deeper into the graph before backtracking
```

The End